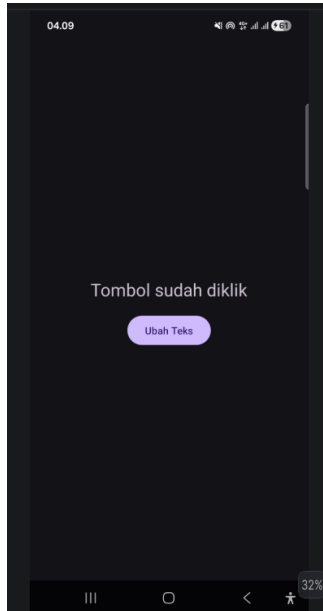


Event, State, dan Implisit Intent pada Jetpack Compose

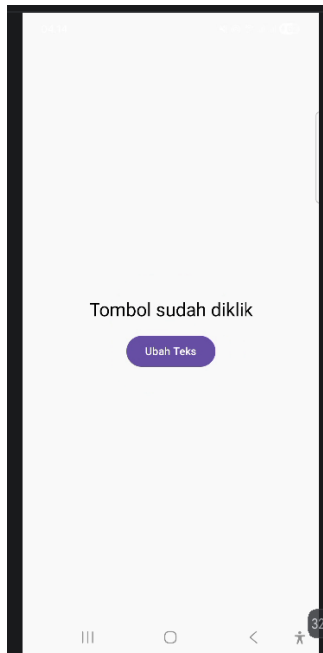
Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Aktivitas Percobaan

1. Projek A



Projek B



2. Objek yang diubah secara langsung adalah objek `TextView` (`tvMessage`).

```
tvMessage.text = "Tombol sudah diklik"[cite: 1]
```

3. Nilai yang diubah adalah nilai variabel *state* `message`

```
message = "Tombol sudah diklik"[cite: 1]
```

4. Karena `jetpack compose` menggunakan paradigma deklaratif. Tampilan ui digambar berdasarkan state saat ini. Pengembang tidak memegang referensi langsung ke objek view untuk mengubah propertinya, melainkan hanya perlu mengubah nilai state. Framework `compose` yang akan memperbarui ui secara otomatis melalui proses `recomposition`.

5. Penjelasan alur:

- User melakukan interaksi/klik tombol.
- Program mencari dan mengakses referensi objek.
- Program secara langsung mengganti properti text pada objek tersebut sehingga layar diperbarui.

6. Alur perubahan:

- User melakukan interaksi/klik tombol.
- event handler mengubah nilai variabel.
- Perubahan state memicu `jetpack compose` untuk menjalankan ulang fungsi `composable` terkait.
- `Composable` menggambar ulang UI dengan nilai state yang baru.

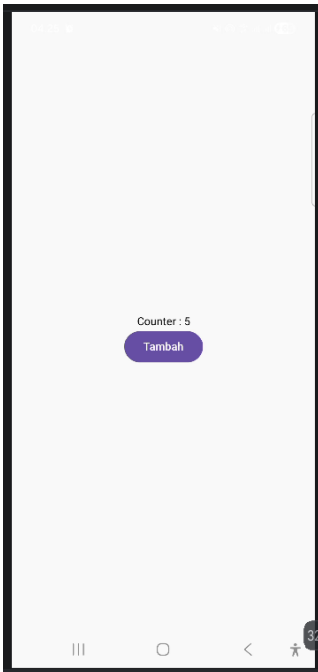
7. Teks pada layar tidak akan berubah saat tombol diklik, meskipun nilai variabel `message` di dalam memori sebenarnya sudah berubah. Penyebabnya adalah variabel biasa tidak dapat diamati oleh `jetpack compose`, sehingga perubahannya tidak akan memicu proses `recomposition`.

8. Cara berpikir imperatif berfokus pada "Bagaimana cara mengubah UI secara langsung step-by-step". Sedangkan pada cara berpikir deklaratif berfokus pada "Bagaimana bentuk UI untuk state tertentu".

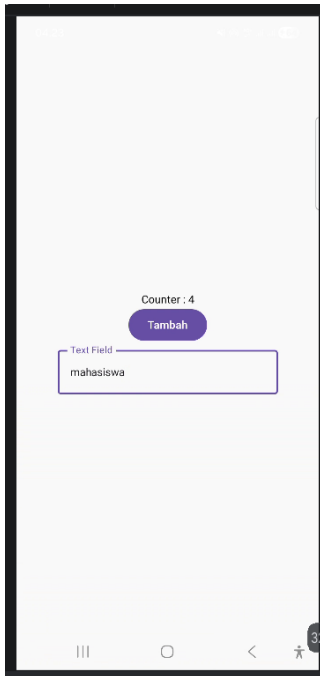
Event Handling State

Aktivitas Percobaan

2. LogCat tetap mencetak pesan log bahwa nilai count bertambah.
3. Tampilan UI pada aplikasi tidak berubah sama sekali.
4. Karena `count` adalah variabel lokal biasa, bukan merupakan objek state. `Compose` tidak mengetahui bahwa ada perubahan data, sehingga tidak menjalankan `recomposition` untuk memperbarui UI.



- 8.
9. LogCat mencetak teks log perubahan nilai count yang bertambah secara berurutan.
10. Angka pada tampilan UI langsung bertambah dan diperbarui sesuai dengan nilai count terbaru.
11. Penggunaan mutableStateOf membuat variabel menjadi observable state yang dipantau oleh Compose. Fungsi remember menjaga agar nilai state tidak ter-reset ke 0 saat recomposition terjadi. Ketika nilainya berubah, Compose mendeteksi perubahan tersebut dan secara otomatis memicu recomposition untuk memperbarui UI.
12. Recomposition adalah proses di mana Jetpack Compose menjalankan kembali fungsi-fungsi @Composable yang terpengaruh oleh perubahan state untuk memperbarui dan menggambar ulang tampilan UI sesuai dengan data terbaru.
- 13.



- 14.
15. LogCat mencetak log berisi teks terbaru sesuai huruf yang dimasukan.
16. Karakter/huruf yang diketik pengguna secara langsung muncul di dalam komponen OutlinedTextField.
17. Setiap kali pengguna mengetik, event `onValueChange` dipicu dan mengubah nilai state `text`. Perubahan state `text` memicu recomposition sehingga OutlinedTextField digambar ulang dengan menampilkan isi `value = text` yang baru.
18. State yang dibuat:
 - State `count` (`var count by remember { mutableStateOf(0) }`)
 - State `text` (`var text by remember { mutableStateOf("") }`)
19. Event yang dibuat:
 - Event `onClick` pada tombol "Tambah".
 - Event `onValueChange` pada OutlinedTextField.
20. Event dijalankan ketika:
 - Event `onClick` dijalankan ketika pengguna menekan/mengklik tombol "Tambah".
 - Event `onValueChange` dijalankan setiap kali pengguna memasukkan, mengubah, atau menghapus karakter/huruf pada OutlinedTextField.

Implisit Intent

Aktivitas Percobaan

11. Fungsi masing-masing action:
 - `Intent.ACTION_VIEW`: Digunakan untuk menampilkan data kepada pengguna. Fungsinya menyesuaikan skema URI yang diberikan, misalnya membuka halaman web (HTTP/HTTPS) di browser atau menampilkan lokasi peta (geo).

- Intent.ACTION_DIAL: Digunakan untuk membuka aplikasi telepon (dialer) dan mengisikan nomor telepon secara otomatis tanpa langsung melakukan panggilan.
 - Intent.ACTION_SENDTO: Digunakan untuk mengirimkan pesan/data ke tujuan tertentu, umumnya khusus untuk mengarahkan pengguna ke aplikasi email berdasarkan skema URI mailto: .
16. Data pada Intent berfungsi untuk memberikan informasi spesifik yang menjadi objek atau target yang akan diproses oleh aksi (*Action*) Intent tersebut. Misal: memberi tahu nomor mana yang harus didial, URL mana yang harus dibuka, atau alamat koordinat mana yang harus ditampilkan.
17. Sistem Android mencocokkan pasangan Action dan Data (URI Schema) dengan deklarasi <intent-filter> dari aplikasi-aplikasi yang terinstall di perangkat. Kombinasi keduanya menentukan jenis aplikasi apa yang mampu menangani permintaan tersebut.